

CONTINUITY: Security-Context Contracts for Composable LLM Agent Controls

Chris Zheng

Co-Founder & Chief Researcher

ZAST.AI

chris@zast.ai

Geng Yang

Co-Founder & CEO

ZAST.AI

geng@zast.ai

September 5, 2026

Artifact: github.com/zast-ai/continuity | License: CC BY 4.0

Abstract

Tool-using large language model agents increasingly combine provenance tracking, task-scoped authorization, policy gateways, protocol adapters, and effect-bound execution. These controls are commonly specified and tested in isolation. Their composition can nevertheless fail when a boundary drops a source label, accepts a self-declared authority root, widens a delegation, changes an approved field without a valid transformation relation, or executes a stale or replayed permit. We call this failure class *security-context discontinuity*.

We define *end-to-end consequence integrity* (ECI): every externally realized effect must have a verifiable witness connecting the exact effect to an authenticated chain origin, principal, task, field-level provenance, root grant, component contracts, current policy, finality sink, and single-use execution state. We introduce an assume-guarantee contract model and a reference system, CONTINUITY, that enforces signed root grants; role-bound component identities; RFC 6901-style leaf paths; signed provenance and context manifests; bounded, source- and value-bound typed releases; independently verifiable transformation witnesses; and subject-, action-, policy-, revocation-, and replay-bound finality permits.

The 1.5-KLOC trusted-core prototype is evaluated with a deterministic conformance suite comprising 32 cross-layer fault classes, four domains, and 20 parameterized instances per fault-domain pair, plus 700 benign and 200 ambiguous tasks. Across 3,460 scenarios and 24,220 system-scenario runs, CONTINUITY contains all 128 fault-domain classes and commits no harmful effect in 2,560 attack instances, while automatically completing all benign tasks and escalating all ambiguous tasks. The strongest incomplete reference configuration commits a harmful effect in 65.6% of attack instances. Targeted ablations reopen 4–24 fault-domain classes depending on the omitted invariant. Median proof verification is 4.21 ms and median end-to-end transition production, verification, permit issuance, and finality is 7.17 ms on the recorded host. These are deterministic conformance results under an ex-

plicit trusted-computing-base model, not an estimate of real-world attack probability or a claim to solve semantic correctness.

1 Introduction

Large language model (LLM) agents increasingly read attacker-writable content, synthesize plans, call external tools, delegate work, and create effects that outlive the model invocation. Indirect prompt injection demonstrates that data retrieved from a web page, email, document, or tool response can influence an agent toward attacker-chosen actions [1–3]. System-level defenses therefore separate untrusted data from control, constrain tool privileges, track causal support, or interpose policy at execution [4–8].

A deployed agent rarely uses only one control. A typical path contains an ingress authenticator, provenance service, memory layer, policy gateway, protocol adapter, tool server, and final effect sink. Each component may be locally reasonable while the end-to-end path is insecure. Consider an email action whose destination was derived from an untrusted invoice. A provenance layer labels the destination and a policy gateway requires a validated release. An adapter later serializes the action but drops the source binding, rewrites a logical alias, or binds the permit to a different representation. A final sink sees a valid signature and an allowed tool name, yet cannot determine whether the concrete destination is the one the gateway approved. The failure is not necessarily in prompt classification or in either endpoint; it is in the composition.

We call this class of failure *security-context discontinuity*: a security-relevant fact is lost, weakened, reinterpreted, or rebound between controls. Discontinuity has several forms. A chain may begin at a self-declared authority root; a component signing key may be accepted for a role it was never authorized to perform; a release may name a field but fail to bind the source value or validity interval; an adapter may perform an allowed-looking transformation

without proving the declared relation; or a finality sink may omit subject, policy, revocation, or replay checks. Cryptographic signatures alone do not prevent these failures. A signature authenticates who emitted an object, not whether the emitter was authorized for that stage or whether its transformation preserved the upstream security contract.

This paper asks:

Under what explicit conditions do independently useful agent-security controls compose into an end-to-end consequence boundary?

We model the LLM planner as adversarial: it may propose any action, parameters, destination, tool, or natural-language justification. The trusted computing base (TCB) consists of explicitly configured root issuers, provenance and release validators, component-role bindings, the deterministic verifier, a key service, and mediated finality sinks. Our goal is not to prove that the model reasons correctly. It is to ensure that an external effect cannot be committed merely because the model proposed it.

We define *end-to-end consequence integrity* (ECI). Informally, every realized effect must carry a verifiable witness from an authenticated root grant through each authorized transition to a current, single-use, effect-bound permit. The witness includes field-level source commitments, component identities and contracts, any typed release or semantic transformation evidence, and finality state. We then express each control as an assume-guarantee contract:

$$\mathcal{C}_i = (A_i, G_i, P_i, M_i),$$

where A_i are input assumptions, G_i are output guarantees, P_i are fields that must be preserved, and M_i are explicit transformation relations. Safe composition requires that upstream guarantees discharge downstream assumptions and that every changed security field is either preserved or justified by an independently checked relation witness. This follows the broader tradition of compositional specifications [9], but instantiates the interface around agent authority, provenance, action semantics, and finality.

We implement these ideas in CONTINUITY. The prototype uses signed root grants, trusted ingress identities, fixed component-role-contract topology, JSON Pointer leaf paths [10], signed provenance/context manifests, signed typed-release credentials, transformation witnesses, and one-shot finality permits. The adapter performs a real security-relevant transformation in every normal scenario: a logical destination alias is resolved to a canonical sink address. The transformation is accepted only when a trusted directory witness binds the before value, after value, relation, component, contract, task, and expiry. Thus, the evaluation does not obtain security by simply forbidding all changes.

We make five contributions:

1. **Problem and taxonomy.** We identify security-context discontinuity as a distinct composition failure and organize 32 fault classes spanning root trust,

provenance and release, component topology, semantic transformation, policy freshness, finality, and replay.

2. **Formal model.** We define ECI, role-bound transition witnesses, and sufficient composition conditions: authenticated roots, field-level continuity, authority non-amplification, validated transformations, current policy, exact effect binding, and complete mediation.
3. **Reference design.** We present proof-carrying context transitions with signed manifests, bounded typed releases, semantic relation witnesses, and state-revalidated finality permits.
4. **Executable artifact.** We provide a dependency-light Python implementation, 30 regression tests, deterministic fault injection, seven reference configurations, targeted ablations, raw results, and reproducibility scripts.
5. **Evaluation.** Across 128 fault-domain classes and 2,560 parameterized attacks, the full configuration commits no harmful effect, while incomplete compositions fail on 4–128 classes. All 700 benign tasks, including 300 bounded external-data releases and a legitimate destination transformation, complete automatically; all 200 unreleased ambiguous tasks escalate.

The result is deliberately narrower than “solving prompt injection.” A malicious planner may still produce harmful text, bad recommendations, or denial of service. CONTINUITY instead establishes a testable structural claim: if the declared TCB and composition conditions hold, a planner-controlled proposal cannot directly cross the finality boundary without a valid end-to-end witness.

2 Background and Motivation

2.1 From prompt compromise to external consequence

Prompt injection is dangerous when model output is connected to an effectful interface. Benchmarks such as InjecAgent and AgentDojo measure whether attacker-controlled observations can redirect tool-using agents [2, 3]. A model-level compromise and a system-level security failure are related but not identical. The model may propose a forbidden action while a deterministic execution boundary rejects it; conversely, the model may propose an apparently benign action whose parameters are later changed or misinterpreted by an adapter.

We therefore separate three events:

1. *proposal*: the planner emits an action candidate;
2. *admission*: a security control authorizes a canonical action;
3. *effect*: a sink commits a state transition in the outside world.

Our primary metric is harmful effect realization, not whether the planner generated the proposal. The planner is assumed compromised in every attack scenario.

2.2 Useful controls and their interfaces

Current defenses contribute complementary properties. CaMeL extracts trusted control and data flow and uses capabilities to constrain exfiltration [4]. Prompt Flow Integrity combines isolation, secure processing of untrusted data, and privilege guardrails [5]. IsolateGPT separates execution domains [11]; Progent expresses programmable privileges [6]; formal-policy work compiles and enforces runtime policies [12]; and ARGUS audits causal provenance from runtime evidence to proposed actions [7]. CXI binds protected fields, typed releases, exact effects, and invocation authority at an execution boundary [8].

These properties are not interchangeable. A provenance tracker does not itself mediate all sinks. A capability authorizes a subject but may not bind the concrete destination. A finality token may bind an exact action but be issued after a compromised adapter has already discarded the provenance needed for policy. A runtime hook provides an interception point but does not specify which security context must survive across the hook. OWASP’s Agent Control Standard focuses on runtime control hooks and policy decisions [13]; the Model Context Protocol specifies tool discovery and invocation and separately documents security considerations [14, 15]. CONTINUITY is designed as a context and contract layer that can be carried across such interfaces; the artifact itself is protocol-neutral and does not claim production integration with either standard.

2.3 Why signatures are insufficient

Suppose an adapter signs an output action. Signature verification establishes that the adapter produced that action. It does not establish:

- that the adapter was authorized for this stage or contract;
- that the chain began at an accepted authority root;
- that an input field and output field denote the same value;
- that an altered destination satisfies an approved transformation;
- that a release applies to this source value and task;
- that the permit is still current and unused at the sink.

This distinction parallels software-supply-chain systems that authenticate a sequence of authorized transformations rather than merely signing the final artifact [16, 17]. CONTINUITY similarly authenticates the transition relation, not only each endpoint.

2.4 Security principles

The design instantiates four established principles. First, *complete mediation*: every path in an effect-equivalence class must cross a compatible sink [18]. Second, *least authority*: a component may exercise only a task- and resource-scoped grant, consistent with zero-trust deployment guidance [19]. Third, *canonical binding*: signatures and permits cover a deterministic representation; the prototype uses a restricted deterministic JSON encoding and identifies RFC 8785 as the interoperability target [20]. Fourth, *freshness and audience restriction*: credentials are short-lived, scoped to a particular sink, and replay-protected, consistent with modern authorization guidance [21].

The distinctive question in this work is not whether each principle is valuable, but what must be carried and checked so that the principles continue to hold after several independently implemented controls transform the request.

3 System and Threat Model

3.1 Entities and execution path

A task traverses the following abstract path:

$$\text{Ingress} \rightarrow C_1 \rightarrow \dots \rightarrow C_n \rightarrow \text{Verifier} \rightarrow \text{FinalitySink}.$$

The ingress produces the first signed envelope E_0 . Each component C_i produces a new envelope E_i and a signed transition receipt ρ_i . The verifier admits or rejects the complete bundle. On admission it creates an execution permit π , and a finality sink consumes π to commit an effect and produce an outcome receipt ω .

The LLM planner may run within or alongside one or more components, but it is not part of the TCB. Its natural-language explanation is never treated as authority. The action it proposes is data until a deterministic verifier and sink admit it.

3.2 Adversary

The adversary controls all natural-language and structured content supplied by external web pages, emails, documents, tool responses, and remote agents. It may cause the planner to emit arbitrary operations, arguments, resources, destinations, tool identifiers, delegation requests, retries, or explanations. It may exploit a faulty or compromised non-root pipeline component to sign an invalid envelope or receipt with that component’s key. It may reorder or truncate chain elements, substitute signed objects, replay permits, mutate an action after authorization, invoke an alternate effect path, or present stale policy and revocation state.

The adversary cannot break SHA-256 collision resistance or forge an Ed25519 signature under an uncompromised key [22]. It cannot directly modify verifier or

finality-sink code, bypass all operating-system isolation around the TCB, or compromise every mandatory effect path. These are explicit assumptions, not properties proved by the protocol.

3.3 Trusted computing base

The TCB contains:

- the deployment policy naming trusted ingress, root-grant, provenance, context, release, and transformation issuers;
- the mapping from pipeline stage to authorized component identity and contract;
- trusted tool-manifest digests and the current policy state;
- deterministic field predicates, transformation relations, and canonicalization;
- the continuity verifier, permit issuer, revocation state, nonce/idempotency ledger, and mediated finality sinks;
- cryptographic keys and their provisioning, rotation, and revocation mechanisms.

A key is trusted only for its configured role. Possession of a registered component key does not authorize that key to originate a chain, issue a root grant, validate provenance, or sign a release. Conversely, compromise of a key accepted as a chain origin or root authority defeats the guarantees rooted in that key. Monotonicity is relative to an authenticated origin; it cannot repair a maliciously authorized root. Compromise of a stage key is narrower: the attacker can impersonate that stage, but a valid chain must still satisfy role, predecessor, contract, transformation, root, finality, and other independently enforced checks. The security analysis in [section 9](#) makes these dependencies explicit.

3.4 Effect model

An action a is a structured object containing an operation, tool and server identity, manifest digest, function, resource, destination, parameters, effect class, and data classification. A sink interprets a and may realize an external effect e . Two syntactically different actions can be effect-equivalent:

$$a \equiv_e a' \quad \text{iff} \quad \text{Effects}(a) = \text{Effects}(a').$$

For example, email, HTTP, browser, shell, and remote-agent interfaces may all exfiltrate the same data. Complete mediation must cover the effect class, not only one tool name.

3.5 Security objective and non-goals

Our objective is structural consequence integrity: a harmful proposal must not become an externally committed

effect without a complete, current authorization witness. We do not attempt to prove that:

- a trusted user intended a semantically correct high-level task;
- a trusted source, release validator, or directory returned true information;
- the model’s prose is accurate, non-toxic, or free from data leakage before a sink;
- arbitrary covert channels, denial of service, or compromised operating systems are prevented;
- the deterministic benchmark estimates a population-level attack probability.

The model is deliberately fail-closed. Missing evidence results in rejection or explicit escalation rather than inferred authority.

4 Security-Context Discontinuity

Definition 1 (Security-context discontinuity). *Let a security-relevant request traverse representations $E_0, \dots, E_n$. A discontinuity exists when a fact required to justify the realized effect is (i) absent, (ii) weakened, (iii) reinterpreted under a non-equivalent schema, (iv) modified without an authorized relation, or (v) no longer fresh at the point of effect.*

A discontinuity can arise even when every object is signed. The signature may belong to an unauthorized role; the object may have a valid signature but an invalid relationship to its predecessor; or the permit may bind an approval-time representation that differs from the sink-time action.

4.1 Minimal counterexample

Assume a policy gateway approves:

operation	=	payment.transfer
resource	=	account:17
destination	=	alias:merchant:9
amount	=	5000
source	=	verified-invoice:44

An adapter resolves the alias and emits a signed action. If it outputs `bankacct:attacker`, then the final signature authenticates the adapter but does not show that the alias resolution is valid. A name-based tool allowlist also passes because the operation remains `payment.transfer`. An action-bound permit issued *after* the adapter likewise binds the attacker’s action perfectly. The missing property is an authenticated relation between the approved input field and executed output field.

CONTINUITY requires a transformation witness w and, independently of the adapter, evaluates the consistency predicate:

$$M_{\text{alias}}(v_{\text{in}}, v_{\text{out}}, w) = \text{true}.$$

The witness binds the path, both value digests, trusted directory issuer, component, contract, task, and expiry. The directory issuer and the semantics of its signed mapping are part of the TCB: the verifier checks that the attested mapping satisfies the configured relation, not that it can discover the real-world alias mapping on its own. A compromised adapter cannot create a valid directory witness.

4.2 Fault taxonomy

The conformance suite instantiates the 32 classes in [table 1](#). The classes are not claimed to be exhaustive. They are designed to exercise distinct proof obligations and common boundary failures.

4.3 Four discontinuity operators

We use four operators to reason about the taxonomy.

Truncation. A projection $q(E)$ omits a required field. If a downstream decision distinguishes two states only through that field, then q makes them observationally equivalent.

Amplification. An output contains authority, delegation scope, or declassified data not justified by the input plus an explicit grant. Natural-language derivation is not a grant.

Rebinding. An approval, release, or permit is reused for a different principal, task, source value, field, tool, destination, action, audience, or policy epoch.

Staleness or replay. An otherwise valid object is used after expiry, revocation, policy change, nonce consumption, or an already-committed idempotency key.

4.4 Why local checks miss the problem

A gateway sees E_i and a sink sees E_j . If no authenticated transition links them, the sink cannot distinguish a valid transformation from a malicious one. Re-signing at every hop does not solve this: a faulty component can sign an arbitrary output. Safe composition therefore needs both endpoint authenticity and a checked transition relation. The transition relation includes structural preservation, semantic transformation predicates, role authorization, and assumption-guarantee compatibility.

5 Formal Model

5.1 Envelopes, roots, and paths

A security-context envelope is

$$E = (K, A, R, s, q, \sigma),$$

where K is the context, A the structured action, R a protocol-specific representation, s the producer identity, q a sequence number, and σ a signature over the canonical unsigned envelope. Security fields are addressed by canonical paths $f \in \mathcal{F}$; the implementation uses JSON Pointer paths such as `/action/parameters/amount_cents`. Let $E[f]$ denote path resolution and $\Delta(E, E')$ the set of leaf security paths whose values differ.

A signed root grant is

$$\Gamma = (p, a, t, U, S, T, V, B, \Pi, P, C, n, x, \sigma_\Gamma),$$

where p is the principal, a the agent actor, t the task root, U an authority set, S a delegation scope, T allowed tool/server/effect identities, V a maximum data classification, B bounded field predicates, Π the policy identity/digest/epoch, P and C provenance and context commitments, n a nonce, and x expiry. A deployment accepts only configured ingress identities and grant issuers.

Definition 2 (Authenticated origin). $\text{RootOK}(E_0, \Gamma)$ holds iff the envelope and grant signatures verify; their producers have the configured ingress and grant-issuer roles; identity, task, grant identifier, policy, provenance, context, and nonce bindings agree; E_0 's authority and scope are subsets of Γ ; its tool, server, effect class, and data class are permitted; every root field predicate holds; and neither object is expired or revoked.

This definition prevents the circular argument that “authority never increased relative to an arbitrary E_0 .” Monotonicity begins only after a separately authenticated root grant.

5.2 Provenance and releases

A provenance manifest contains source records (id, c, h) and claims (f, id, h_v) , binding path f to a source identity and the digest of the value at $E_0[f]$. Its signed digest is P . A context manifest commits to the source-classified items visible to the planning context; its signed digest is C . The verifier recomputes both manifest digests, verifies issuer roles and signatures, resolves every claimed path, and checks each leaf-value digest.

Untrusted sources have classes EXTERNAL, TOOLRESULT, or MEMORY. Such a value may influence a protected action field only through a release credential

$$L = (p, a, t, P, id_s, h_s, f, h_v, \phi, op, tool, n, x, \sigma_L),$$

which binds the principal, actor, task, provenance manifest, source identity and digest, target path, exact value digest, bounded predicate ϕ , action operation, tool, nonce, expiry, and trusted release issuer.

Definition 3 (Valid typed release). $\text{ReleaseOK}(L, E_0, M)$ holds iff L has a trusted, valid, non-revoked signature; all identity, task, manifest, source, path, operation, and tool bindings match; $H(E_0[f]) = h_v$; the source claim in M has the same value digest; $\phi(E_0[f])$ is true; and L is unexpired.

Table 1: Fault classes in the conformance suite. Each class is instantiated in four domains.

Family	Fault classes	Violated obligation
Root and grant	Untrusted root producer; root authority exceeded; root scope exceeded; root field-constraint bypass	Accepted ingress/issuer, authenticated root grant, bounded operation/resource/field constraints
Provenance and release	Untrusted field binding; provenance drop; provenance value substitution; memory laundering; release predicate bypass; release value substitution; expired release	Signed manifest, leaf-value binding, source continuity, source/value/path/predicate/issuer/expiry-bound release
Identity and topology	Principal substitution; unauthorized stage signer; receipt/producer mismatch	Principal continuity, stage-identity-contract binding, authorized predecessor and receipt producer
Authority and policy	Authority amplification; delegation widening; taint downgrade; policy downgrade; context-root omission	Non-amplification, scope monotonicity, taint monotonicity, policy freshness, model-context commitment
Action semantics	Argument mutation; destination substitution; tool-server swap; effect-class downgrade	Leaf preservation, exact tool/manifest/server binding, exact effect classification
Transform contracts	Missing transform witness; invalid transform witness; contract-guarantee violation	Declared relation, trusted witness, before/after binding, postcondition satisfaction
Finality state	Subject substitution; post-permit action substitution; revoked grant	Caller binding, exact action binding, finality-time policy/revocation recheck
Lifecycle and mediation	Nonce replay; retry duplication; alternate path	One-shot nonce, idempotency, complete effect mediation

A release is therefore not a field-name whitelist. It authorizes one validated source value to influence one named target field under one bounded predicate and task context.

5.3 Component contracts

A component contract is

$$\mathcal{C}_i = (F_i^{req}, Q_i^{req}, A_i, G_i, F_i^{ens}, Q_i^{ens}, P_i, M_i),$$

where F^{req} and F^{ens} are required and ensured fields; Q^{req} and Q^{ens} are deterministic predicates on input and output envelopes; A_i are required upstream guarantee tags; G_i are guarantee tags produced on success; P_i is a set of path roots that must be preserved; and M_i maps transformable paths to independently evaluated relations.

A deployment also fixes a pipeline binding

$$B_i = (stage_i, role_i, signer_i, contract_i).$$

Thus, a key valid for one role is not accepted for another stage or contract.

A transition receipt is

$$\rho_i = (stage, role, id_C, H(\mathcal{C}_i), H(E_i), H(E_{i+1}), \Delta_i, W_i, Q_i^{req}, Q_i^{ens}, signer, \sigma_i).$$

For a changed path f , a transformation witness $w \in W_i$ binds the relation identifier, f , both value digests, principal, task, component signer, contract, parameters, expiry, and an independently trusted validator signature.

Definition 4 (Valid transition). $StepOK(E_i, \rho_i, E_{i+1}, \mathcal{C}_i, B_i)$ holds iff:

1. the envelope and receipt signatures, input/output digests, sequence, and recomputed change set agree;

2. the receipt signer equals the output producer and matches B_i 's authorized stage, role, signer, contract, and predecessor;
3. all required fields, required guarantee tags, and input predicates hold;
4. for every changed security path f , either f is not in a preserved root and a declared relation $M_i(f)$ holds with a valid witness, or the step is rejected;
5. authority and delegation do not widen, taint does not decrease, and policy epoch does not decrease except through a separately authorized policy transition;
6. all ensured fields and output predicates hold, after which G_i becomes available downstream.

In the prototype, policy changes are not a permitted component transformation; any changed policy identity, digest, or epoch must still equal current deployment state at admission and finality.

5.4 Permits and finality

After verifying the bundle, the verifier issues

$$\pi = (p, a, t, \Gamma, sink, H(A_n), H(bundle), \Pi, n, k, x, one, \sigma_\pi),$$

where k is an idempotency key. A finality sink validates the permit issuer, caller subject, audience, exact action digest, current policy identity/digest/epoch, current revocation state, expiry, nonce, and idempotency state before committing an effect.

Definition 5 (Effect witness). An effect witness is

$$W_e = (\Gamma, M_P, M_C, L^*, E_0, \rho_0, E_1, \dots, \rho_{n-1}, E_n, \pi, \omega),$$

including the root grant, provenance and context manifests, any releases and transformation witnesses, the transition chain, permit, and outcome receipt.

Definition 6 (End-to-end consequence integrity). *A system satisfies ECI iff for every externally realized effect e , there exists W_e such that*

$$\text{Verify}(W_e) = 1 \quad \wedge \quad \text{Realize}(\text{Canon}(W_e)) = e,$$

and the witness's authorization was current and unconsumed immediately before e was committed.

5.5 Composition conditions

For a pipeline of n components, the sufficient conditions are:

C1 Authenticated origin.

$\text{RootOK}(E_0, \Gamma)$ and authenticated provenance/context commitments hold.

C2 Contract compatibility.

Each required guarantee of $\mathcal{C}_{i+1}$ is established by the root or a valid predecessor, and all deterministic assumptions and postconditions are checked.

C3 Context continuity.

Every security-critical changed path is preserved or justified by a declared, independently verified relation witness.

C4 Non-amplification.

Authority and delegation can increase only through a separately authenticated grant, data taint does not decrease across ordinary transitions, and any protected field sourced from untrusted data requires a valid typed release bound to the source value and target field.

C5 Canonical action binding.

The permit and sink bind the same canonical action, including tool/server manifest, operation, resource, destination, parameters, effect class, and data class.

C6 Fresh finality.

The sink rechecks subject, audience, policy, revocation, expiry, nonce, and idempotency immediately before effect.

C7 Complete mediation.

Every path capable of realizing an effect in the protected equivalence classes traverses a compatible finality sink.

Theorem 1 (End-to-end composition safety). *Assume unforgeability of the signature scheme and collision resistance of the digest. If C1–C7 hold for a task, then every realized protected effect has a valid effect witness and the system satisfies ECI, even if the planner and all attacker-writable content are adversarial.*

The proof proceeds by induction over valid transition receipts, followed by finality binding; [section A](#) gives the full argument.

Theorem 2 (Context truncation). *Let a downstream decision D depend on security field f . If an intermediate projection q omits f , then there exist envelopes E, E' with $q(E) = q(E')$ but $D(E) \neq D(E')$. A downstream component lacking authenticated recovery of f must either fail closed or risk accepting an invalid state.*

Theorem 3 (Unbound-field substitution). *If a security-critical field f affects the realized effect but is not jointly bound by admission and finality, then an adversary controlling the intervening representation can substitute $A[f]$ with $A'[f] \neq A[f]$ while preserving all checked permit fields.*

Theorem 4 (Authority non-laundering). *If every ordinary derivation step preserves or reduces authority and only a verified grant edge may add authority, then no finite sequence of LLM generation, summarization, memory consolidation, or protocol translation can create execution authority absent from the authenticated root and grants.*

In CONTINUITY, this premise is enforced by construction of **StepOK**: an ordinary transition is accepted only when authority is preserved or reduced, while an increase requires a separately verified grant edge.

Proposition 1 (Incomplete mediation). *If a protected effect class contains any reachable path that bypasses a compatible finality sink, then the system does not satisfy ECI for that class, regardless of the correctness of the mediated path.*

6 Design

6.1 Authenticated chain origin

The deployment policy names accepted ingress identities and root-grant issuers separately. The first envelope must be signed by a trusted ingress, while the root grant must be signed by a trusted authority. The verifier checks that the root's identity, task, authority, delegation, tool/server, effect class, policy, provenance/context commitments, field constraints, nonce, expiry, and revocation state agree with the grant.

This two-role design prevents two attacks from the first artifact. First, a non-ingress component key cannot create a self-signed E_0 and then sign the rest of the chain. Second, E_0 cannot simply declare a large authority set and rely on downstream monotonicity. The accepted origin is bounded by a separately authenticated grant.

Field constraints are deterministic predicate specifications. For example, a finance task may permit `/action/parameters/amount_cents` only in $[1, 10^6]$, while an external-data release for the same field is more restrictive, such as $[1, 2 \cdot 10^4]$. The root grant bounds the overall task; the release bounds the particular untrusted value.

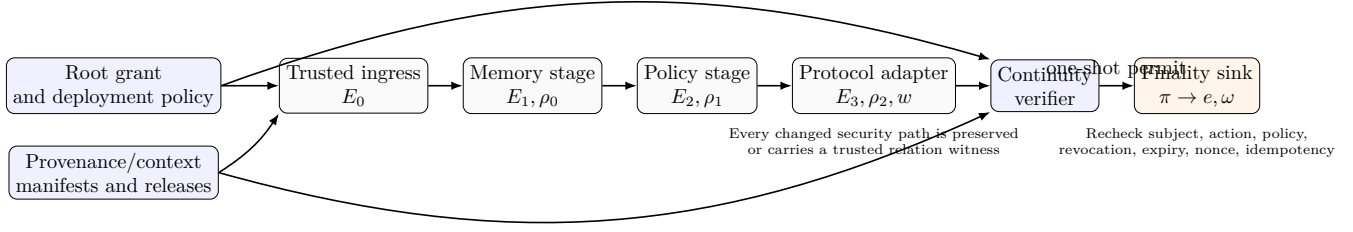


Figure 1: CONTINUITY architecture. The planner is outside the trusted path and may propose arbitrary actions. Security comes from authenticated roots, proof-carrying transitions, and a revalidating finality boundary.

6.2 Field-resolved provenance

The provenance manifest contains source records and path claims. A claim binds

$$(path, source_id, H(value)).$$

The verifier parses the path, resolves the actual leaf in E_0 , and recomputes the digest. Thus, `/action/parameters/amount_cents` is not an opaque label. A manifest that claims a source for one amount but accompanies another amount fails before transition verification.

The context manifest similarly commits to the source-classified items supplied to the planning context. In this prototype, the verifier authenticates the signed manifest and its root; the surrounding runtime is responsible for ensuring the manifest actually covers the bytes shown to the model. This trust boundary is stated explicitly in [section 12](#).

6.3 Bounded typed releases

Untrusted content is data, not authority. Nevertheless, useful tasks often need external values to determine an action field: an invoice amount, a deployment commit, a recipient alias, or a delegated capability. A blanket rule preventing all such influence would destroy utility.

A typed release solves this by binding one source value to one target path. The credential includes source and manifest digests, exact value digest, principal, actor, task, operation, tool, predicate, issuer, nonce, and expiry. The verifier checks all fields and evaluates the predicate on the value resolved from E_0 . Examples in the artifact are:

- finance: integer amount in [1, 20,000] cents;
- workspace: destination alias with prefix `alias:workspace:customer::`;
- DevOps: exact signed 40-hex commit identifier;
- delegation: capability in the enum `{report.generate}`.

A release signed for value v cannot be replayed for v' , even if both satisfy the predicate, because the credential also binds $H(v)$. An expired or revoked credential is rejected.

6.4 Role-bound transition receipts

For each stage, the deployment fixes the expected role, signer, and contract. A receipt commits to the input and output envelope digests, contract digest, recomputed changed paths, assumption and guarantee predicate identifiers, and transformation-witness identifiers. Verification requires:

1. receipt signer equals output producer;
2. signer, role, contract, stage order, and predecessor match deployment;
3. contract assumptions and required upstream guarantees hold;
4. all security changes conform to preservation or a declared relation;
5. contract postconditions hold before its guarantee tags are made available downstream.

A component cannot select an arbitrary trusted contract. The contract is authorized for a specific stage and signer by deployment policy.

6.5 Semantic transformations

Real pipelines transform action representations. Alias resolution, unit conversion, schema translation, and tenant routing may legitimately change a security field. Treating every change as malicious makes the system unusable; treating a path listed in `may_transform` as freely mutable makes the declaration meaningless.

CONTINUITY therefore attaches a relation identifier to each transformable path. For a destination alias, the independently signed witness contains the alias, resolved address, both value digests, trusted directory issuer, component signer, contract, principal, task, and expiry. Independently of the adapter, the verifier evaluates:

$$M_{alias}(v, v', w) \equiv v = w.alias \wedge v' = w.resolved \\ \wedge v \text{ is logical} \wedge v' \text{ is canonical.}$$

It also verifies that the witness issuer is trusted for `alias_resolution`. This establishes consistency with a directory-attested mapping; the trusted directory and

relation registry remain part of the TCB and are not reconstructed by the verifier. Every normal benchmark instance exercises this path. Missing witness, untrusted issuer, false relation, digest mismatch, or postcondition failure rejects the chain.

6.6 Assume-guarantee flow

Contracts expose both structural fields and semantic guarantee tags. The root verifier establishes `root-authenticated`, `provenance-authenticated`, and `context-authenticated`. The memory contract requires those tags and, after its predicates and transition checks pass, establishes `memory-context-preserved`. The gateway requires that guarantee and establishes `policy-authorized`. The adapter requires the gateway guarantee and establishes `canonical-action` only if its destination postcondition and transformation relation hold.

A failed stage does not emit usable downstream guarantees. This prevents a later component from satisfying its assumptions merely because a string label appeared in a receipt.

6.7 Finality and lifecycle state

The verifier issues a permit only after the whole proof bundle passes. The permit binds the principal, actor, task, root grant, sink audience, exact action digest, proof-bundle digest, current policy identity/digest/epoch, nonce, idempotency key, expiry, and one-time flag.

The finality sink receives the actual caller identity and current runtime state. Immediately before effect it rechecks the signature, caller subject, audience, action digest, policy, revocation, expiry, nonce, and idempotency state. A retry with a new nonce but the same idempotency key returns the prior outcome rather than creating a second effect. A permit issued before a grant is revoked is rejected at finality.

6.8 Complete mediation

The proof protects only sinks that require a valid permit. A deployment must enumerate protected effect classes and route every equivalent interface through a compatible broker. The artifact models a direct alternate path as a negative control. Disabling mediation causes the effect even though every receipt and permit mechanism on the normal path remains correct.

7 Reference Implementation

The artifact is a Python 3.11+ reference implementation. The security core is 1,523 lines in `src/continuity/core.py`; scenario construction, fault injection, reference configurations, aggregation, and timing add 1,697 lines in `experiment.py`. The repository

includes 30 regression tests and publication scripts. It does not invoke an LLM: attack candidates are instantiated directly because the planner is adversarial in the model.

7.1 Canonical objects and signatures

All signed dataclasses are serialized to deterministic JSON with sorted keys, integer-only benchmark numerics, normalized sets, and no non-finite floats. SHA-256 supplies object digests, and Ed25519 supplies signatures. This restricted encoder is sufficient for internal reproducibility but is not claimed to be a complete RFC 8785 implementation [20]. Production interoperability requires a standardized canonicalizer, explicit Unicode normalization, key rotation, hardware-backed keys, and certificate or workload-identity validation.

The keyring contains public keys, but trust is role-specific in `DeploymentPolicy`. Merely appearing in the keyring does not authorize a key to act as ingress, authority, provenance issuer, release validator, transformation validator, component stage, verifier, or sink.

7.2 JSON Pointer and leaf changes

Security fields use RFC 6901-style paths [10]. Fixed context and action fields are atomic, while nested action parameters are recursively enumerated at leaf granularity. For example, changing only an amount produces:

```
/action/parameters/amount_cents
```

rather than a change to the undifferentiated parameters object. The same resolver is used for provenance claims, root constraints, release targets, transform rules, and change-set recomputation. Missing paths fail closed.

7.3 Verifier algorithm

The actual verifier accumulates reason codes rather than stopping at the first error. This improves auditability and supports targeted regression tests. Duplicate identifiers, chain-length mismatch, sequence gaps, false change sets, unused or unknown transform witnesses, and undisclosed assumption/guarantee identifiers are also rejected.

7.4 Finality implementation

The permit issuer receives only an allowed decision and the final envelope. The sink independently verifies the permit signature and runtime state. The caller subject is an explicit input to `FinalitySink.execute`; it is not inferred from permit contents. The sink stores consumed nonces and outcome receipts in an in-memory ledger. On an idempotent retry it returns the existing signed outcome without applying the action again. Effects are recorded only in an in-memory simulated world.

```

Verify(bundle, deployment, now):
  check root-envelope and root-grant signatures
  enforce trusted ingress and grant issuer
  bind identity, task, authority, scope, policy,
    provenance/context roots and field bounds

  verify provenance and context manifests
  resolve each claimed leaf and compare value digest
  verify releases and evaluate bounded predicates

  guarantees <- authenticated-root guarantees
  for each deployed stage i:
    check signer/role/contract/order/predecessor
    verify envelope and transition receipt
    recompute changed leaf paths
    check required fields, assumptions, guarantees
    for each changed security path:
      require declared transform relation
      verify trusted witness and relation predicate
    check monotonic authority/scope/taint/policy
    check ensured fields and postconditions
    publish stage guarantees only on success

  check final authority, scope, tool manifest,
    current policy, expiry and destination
  return decision

```

Figure 2: Simplified verification procedure. Every check is deterministic; model-generated explanations are ignored.

7.5 Reference pipeline

The evaluated pipeline has three role-bound stages:

Memory.

Preserves identity, provenance, root, action, and context fields; establishes `memory-context-preserved`.

Gateway.

Requires that guarantee, checks current policy as a deterministic postcondition, and establishes `policy-authorized`.

Adapter.

Requires gateway authorization and resolves the logical destination to a canonical address under an `alias_resolution` relation witness; establishes `canonical-action`.

Authority, delegation scope, taint, and policy fields have independent global monotonicity checks in addition to contract logic. This intentional redundancy explains why disabling one monotonicity check does not always create an attack when another invariant still rejects the same mutation.

7.6 Static contract linting

A lightweight linter checks field availability and guarantee-tag flow before runtime. It ensures that a stage’s required fields and guarantees are available from the authenticated root or predecessors. Runtime verification remains authoritative because static tags alone cannot validate concrete signatures, values, predicates, or transformation relations.

7.7 Artifact organization

The artifact includes raw per-run CSV files, per-fault summaries, ablations, performance measurements, figures, tests, and a claim-to-evidence guide. The public artifact is available at <https://github.com/zast-ai/continuity>. The code is distributed under the MIT License; copyright holders are Chris Zheng, Geng Yang, and ZAST.AI.

8 Evaluation

We ask five research questions:

RQ1.

Do individually plausible but incomplete control compositions prevent harmful effects?

RQ2.

Does the full system contain each modeled discontinuity class across domains?

RQ3.

Can bounded external data and legitimate semantic transformations retain utility?

RQ4.

Which invariants are exposed by targeted ablations?

RQ5.

What latency and proof-size overhead does the reference implementation add?

8.1 Methodology

Domains. The deterministic generator covers workspace email, finance payments, DevOps deployment, and multi-agent delegation. Each action contains domain-specific operations, resources, destinations, parameters, effect classes, and data classifications.

Faults and scale. The suite instantiates the 32 fault classes in [table 1](#) in all four domains, with 20 parameterized instances for each fault–domain pair:

$$32 \times 4 \times 20 = 2,560 \text{ attack instances.}$$

It also contains 400 direct benign tasks, 300 benign tasks using signed typed releases, and 200 ambiguous tasks containing an unreleased external field. The total is 3,460 scenarios. Seven system configurations produce 24,220 system–scenario runs.

The 20 instances vary identifiers, principals, actors, tasks, resources, parameter values, nonces, digests, and signatures, but share a deterministic fault schema. They are not independent samples from a real-world attack distribution. We therefore report exact conformance counts and rates over the generated suite, not binomial confidence intervals or population-level security claims.

Table 2: Main conformance results. “Classes” is the number of contained fault–domain pairs out of 128.

Configuration	Effect ASR	Classes	Benign	Ambig.
Pass-through	100.0%	0	100%	0%
Tool allowlist	100.0%	0	100%	0%
Gateway policy	96.9%	4	100%	0%
Provenance gateway	84.4%	20	100%	100%
Effect-bound permit	81.3%	24	100%	0%
Gateway + finality	65.6%	44	100%	100%
CONTINUITY	0.0%	128	100%	100%

Adversarial planner. Every attack begins after compromise of planning: the candidate action or security transition is directly instantiated in the attacker’s desired form. This avoids crediting the security boundary for an LLM that happens to refuse a payload.

Reference configurations. The six incomplete configurations are mechanism-level negative controls, not reimplementations of named systems:

- *PassThrough*: directly executes the final action;
- *ToolAllowlist*: checks only the final tool identifier;
- *GatewayPolicy*: evaluates authority, scope, policy, and taint at the gateway, then trusts downstream changes;
- *ProvenanceGateway*: additionally validates field provenance and releases at the gateway;
- *EffectBoundPermit*: binds the post-adapter action to finality without validating its upstream justification;
- *Gateway+Finality*: combines gateway policy with an action-bound finality permit, but does not authenticate the gateway-to-adapter transition.¹

Metrics. *Effect attack success rate* (effect ASR) is the fraction of attack instances that commit a harmful effect. For replay and retry faults, failure requires more than one committed effect; a single authorized effect followed by a rejected or idempotent retry is correct. *Benign auto-completion* is the fraction of benign tasks that commit one effect without escalation. *Ambiguous escalation accuracy* is the fraction of unreleased external-field tasks that commit no effect and return ESCALATE. *Lifecycle correctness* compares both expected terminal state and expected effect count, rather than treating every attack as requiring the literal disposition DENY.

8.2 Main results

CONTINUITY commits no harmful effect in 2,560 attack instances and contains every one of the 128 fault–domain classes. It completes all 700 benign tasks and escalates

¹The permit is bound to the final post-adapter canonical action because benign traces include an authorized alias-to-canonical transformation. Binding the permit to the pre-transform gateway representation would reject valid traces rather than isolate the remaining composition gap.

all 200 ambiguous tasks. Its lifecycle correctness is 100%, including replay and retry semantics.

The strongest incomplete configuration, Gateway+Finality, still has 65.6% effect ASR. This result illustrates the central composition claim. Gateway authorization plus exact final action binding is insufficient when the permit issuer accepts a post-adapter action without verifying how it relates to the gateway-approved state. Conversely, EffectBoundPermit protects replay, subject, revocation, and post-permit substitution but lacks root, provenance, and contract justification; its ASR is 81.3%.

GatewayPolicy contains only four fault–domain classes because it observes the pre-adapter state. Provenance-Gateway contains 20 classes and correctly escalates unreleased external fields, but downstream provenance loss, parameter changes, role misuse, and alternate paths remain outside its view.

8.3 Utility and validated change

All normal traces perform a security-relevant adapter transformation. The ingress and gateway carry a logical destination alias; the final action carries a canonical address. The full system accepts the transformation only with a trusted directory witness and verified alias relation. Thus, 100% benign completion does not result from a preserve-everything contract.

All 300 external-data tasks carry a signed typed release. The verifier checks source identity and digest, leaf path, exact value digest, operation, tool, bounded predicate, issuer, task, expiry, and revocation. All 300 complete. In contrast, all 200 otherwise plausible tasks that omit a release return ESCALATE and commit no effect. The release attack classes demonstrate negative controls: an out-of-bound value, value substitution, or expired credential is rejected even when the target field name is present.

8.4 Ablation study

The ablation suite uses one representative instance for each of the 128 fault–domain pairs. table 3 reports reopened classes. “NoContractConformance” retains envelope and receipt signatures, digest links, recomputed change sets, global monotonicity, and finality; it disables preservation, transform-rule, and contract postcondition checks. It is therefore not mislabeled as removal of receipts.

The three largest effects are field-provenance removal, contract-conformance removal, and incomplete mediation, each reopening 24 classes. Root authentication reopens 16, including the exact self-origin and over-broad-origin counterexamples. Release validation reopens 12 classes; transform validation and replay protection each reopen eight. Authority monotonicity alone reopens none in this benchmark because operation and action changes remain rejected by root, preservation, scope, and exact-action controls. This demonstrates defense in depth and also

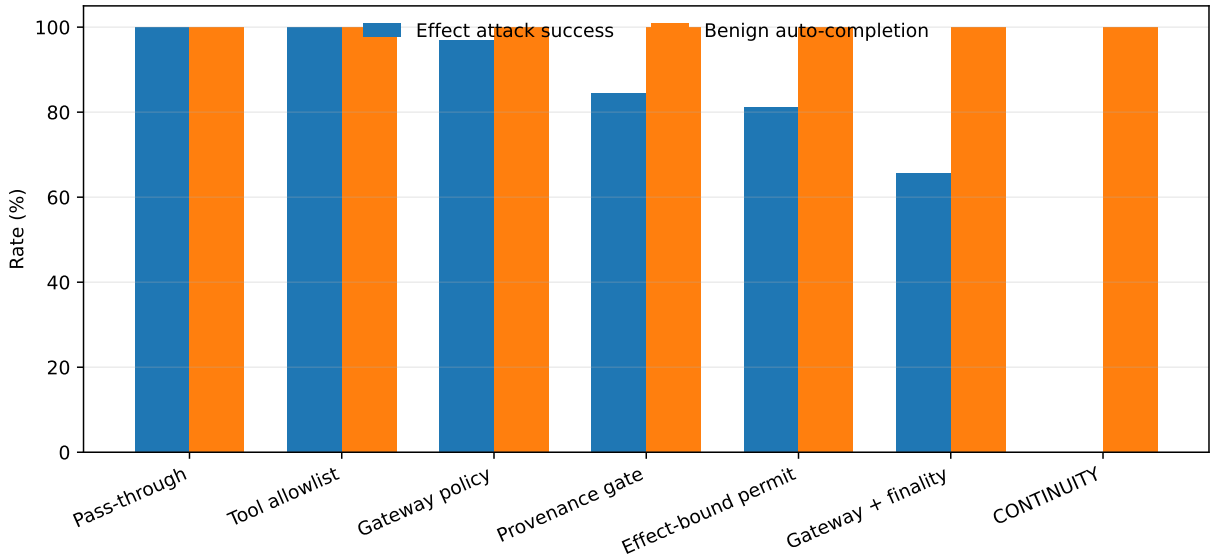


Figure 3: Harmful-effect success and benign auto-completion. The full system preserves benign completion while incomplete compositions remain vulnerable to cross-layer faults.

Table 3: Targeted ablations over 128 fault-domain classes. Zero for one check can indicate redundant containment by another invariant, not irrelevance to the sufficient-condition theorem.

Ablation	Reopened classes
No field provenance	24
No contract conformance	24
Incomplete mediation	24
No root authentication	16
No release validation	12
No transform-witness validation	8
No replay protection	8
No component-role binding	4
No identity binding	4
No delegation monotonicity	4
No taint monotonicity	4
No policy freshness	4
No context commitment	4
No action binding	4
No subject binding	4
No revocation recheck	4
No authority monotonicity alone	0

cautions against interpreting an ablation as proof that a theorem condition is logically unnecessary.

8.5 Performance

Measurements were collected on the recorded Linux host using Python and the `cryptography` Ed25519 implementation. Each reported microbenchmark operation is repeated 300 times. Timing is host- and runtime-dependent and excludes network, model, external-policy-service, and durable-storage latency.

Verification scales approximately linearly with the number of signed transitions. In the recorded run, bundles with 1, 3, 5, 10, and 20 transitions occupy approximately 8.1, 12.4, 16.8, 27.6, and 49.4 KiB, respectively. The implementation stores full signed envelope snapshots. A production

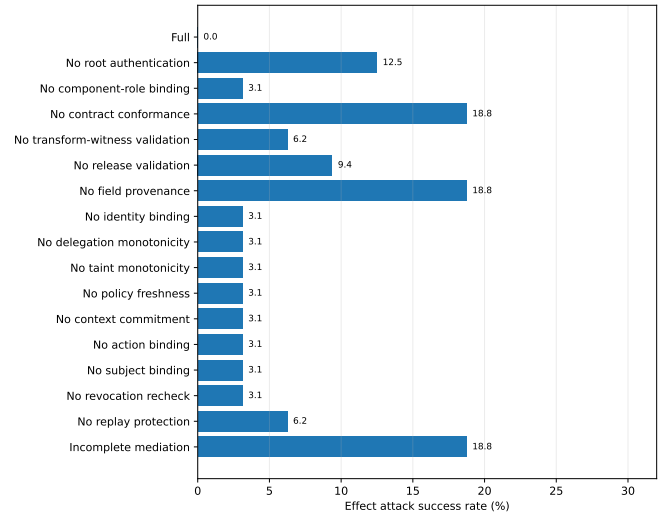


Figure 4: Effect ASR under nonzero targeted ablations. Full CSV results retain the zero-valued redundant ablations.

encoding could use deltas, Merkle commitments, checkpoint receipts, batch verification, and compact binary serialization.

8.6 Interpretation

The evaluation supports a conformance statement: for every generated instance, the full implementation enforces the modeled obligations, and removing particular obligations exposes corresponding faults. It does not establish a universal attack rate, completeness of the fault taxonomy, correctness of trusted validators, or resistance to a compromised TCB. Those limitations are addressed in

Table 4: Reference-prototype latency.

Operation	p50	p95
Proof verification	4.21 ms	4.91 ms
End-to-end transition + finality	7.17 ms	8.07 ms

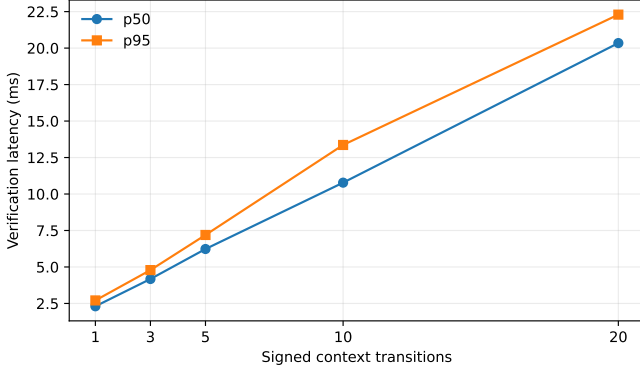


Figure 5: Proof-verification latency versus signed transitions.

section 12.

9 Security Analysis

9.1 Theorem-to-implementation correspondence

Theorem 1 is conditional. The artifact now enforces each stated condition rather than treating it as an external assumption without a check:

- C1 maps to trusted ingress, signed root grant, role-specific issuer sets, root field constraints, signed provenance/context manifests, and leaf-value digest checks.
- C2 maps to pipeline stage bindings, required/ensured guarantee tags, and deterministic input/output predicates.
- C3 maps to JSON Pointer leaf changes, preserved roots, declared transform rules, and signed relation witnesses.
- C4 maps to root subsets, transition monotonicity, and source/value/path/predicate-bound releases.
- C5 maps to a complete structured action digest shared by verification and finality.
- C6 maps to finality-time subject, policy, revocation, expiry, nonce, and idempotency checks.
- C7 maps to the permit-requiring effect broker; the alternate-path fault demonstrates the consequence of violating it.

The implementation does not mechanically prove that its Python code refines the formal model. The regression suite supplies executable evidence for the specified

checks and counterexamples; a proof assistant or verified implementation remains future work.

9.2 Root compromise

Monotonicity cannot secure an attacker-chosen origin. If a key accepted as a root-grant issuer is compromised, the attacker can issue an over-broad root grant; if both deployment trust and revocation fail to remove that key, the guarantee is lost. Similarly, compromise of a trusted provenance, release, or transformation validator allows false claims within that validator’s role.

This is different from compromise of an ordinary pipeline component. A memory, gateway, or adapter key is authorized only at its configured stage and contract. The attacker can sign that stage’s output, but cannot start a chain, issue a grant, impersonate a different stage, create a trusted release, or create a trusted transformation witness. It may still exploit any overly permissive relation or predicate configured for its contract. Thus, stage-key compromise is contained only to the extent that independently checked contracts and validators are restrictive.

9.3 Signatures versus authorization

The verifier distinguishes key presence from role authorization. Three checks are separate:

$$\begin{aligned} &\text{SigValid}(k, o), & \text{RoleAllowed}(k, r), \\ &\text{TransitionOK}(o_{in}, o_{out}, \mathcal{C}). \end{aligned}$$

An object is accepted only when all required predicates hold. This prevents the attack in which a valid adapter key signs E_0 and every receipt, or a receipt is signed by a different valid component than the output producer.

9.4 Typed-release safety

A release predicate limits the allowed value set, but predicate satisfaction alone is insufficient. Without an exact value digest, a credential could be replayed for another value that also satisfies the predicate. Without source and manifest binding, the same value could be attributed to a different document. Without principal, task, operation, and tool binding, it could be replayed across workflows. The implemented release checks all of these dimensions.

Trusted-release issuance remains security-critical. CONTINUITY verifies that an issued credential is internally and contextually valid; it does not prove that the validator’s extraction from an invoice or other source was semantically correct.

9.5 Transformation safety

A contract declaration that a path “may transform” is not itself authorization. The relation witness must be signed by an issuer trusted for that relation and bind both before and after values. The verifier independently

evaluates the relation predicate. The current artifact implements alias resolution. A production system would require relation-specific validators for currency conversion, schema normalization, tenant routing, or identifier resolution, together with versioned semantics and revocation.

If the relation predicate is non-injective, omits a security field, or admits attacker-chosen outputs, [theorem 3](#) applies. The relation registry is therefore part of the TCB.

9.6 Freshness and time-of-check/time-of-use

Admission-time correctness is not finality-time correctness. A policy can change, a grant can be revoked, or the caller can be substituted after a permit is issued. The sink rechecks the current state rather than relying solely on the verifier’s earlier decision. It also atomically consumes nonce/idempotency state before effect in the model.

A real distributed provider may have its own concurrency, partial-failure, and idempotency semantics. The prototype’s in-memory atomicity is a simplifying assumption. Production adapters need provider-specific transactional or compensating behavior.

9.7 Complete mediation

The system cannot protect a path it does not mediate. Tool aliases, direct SDK calls, browser automation, shell commands, remote agents, and recovery paths must be mapped to effect classes and routed through compatible sinks. Deployment review should therefore test effect reachability, not only registered tool names. The artifact’s `ALTERNATE_PATH` fault commits an effect immediately when mediation is disabled, even though all normal-path cryptographic checks remain intact.

9.8 Residual attacks

An adversarial planner can still cause denial of service by repeatedly proposing rejected actions, consume policy and signature-verification resources, or produce unsafe textual output not connected to a protected sink. A malicious but trusted source may supply incorrect facts that satisfy all structural checks. Covert exfiltration through allowed text, timing, resource names, or aggregate queries is not eliminated. `CONTINUITY` is a consequence-integrity boundary, not a complete information-flow or semantic-safety solution.

10 Discussion

10.1 What should be standardized?

The artifact suggests that interoperability requires more than a universal `ALLOW/DENY` hook. A cross-control contract needs stable semantics for at least:

- principal, actor, task, root grant, and delegation scope;

- field-resolved provenance and model-context commitments;
- canonical action identities and protected effect classes;
- typed releases and transformation-relation witnesses;
- policy identity/digest/epoch, audience, nonce, idempotency, and outcome.

MCP and runtime-hook standards can carry such objects, but the paper does not propose replacing their transport or authorization layers. The likely integration point is an extension or sidecar envelope associated with tool calls and agent handoffs. An implementation must specify canonicalization and field paths identically at the producer and verifier.

10.2 Fail closed versus escalation

Missing proof does not always mean the user requested a forbidden task. An unreleased invoice value may be legitimate but insufficiently validated. The system therefore distinguishes `ESCALATE` from `DENY`. Escalation can request a signed source validator, narrower user confirmation, reauthentication, or human approval bound to the exact action digest. Invalid signatures, role violations, root excess, stale state, and false relation witnesses remain hard failures.

10.3 Trust distribution

The prototype stores deployment trust locally. In a single enterprise, ordinary PKI, workload identity, or a transparency log may be sufficient. Cross-organization agent interactions may require shared issuer discovery, revocation, and evidence anchoring. A consortium or public ledger could anchor policy, manifest, revocation, or batch-evidence roots, but it is optional. The safety property comes from authenticated contracts and finality enforcement, not from placing prompts or private task data on a blockchain.

10.4 Policy and relation evolution

Contracts, predicates, and relation semantics are versioned security code. Updating `alias_resolution` or a release predicate changes the accepted effect set and should trigger review, compatibility testing, and policy-epoch changes. A permit issued under an older policy is rejected once the sink’s runtime state advances. Long-lived workflows need explicit migration or reauthorization rather than silent compatibility.

10.5 Observability and incident response

A complete witness supports questions that ordinary traces cannot reliably answer: Which root grant authorized the task? Which source supplied a protected field? Which release and predicate admitted it? Which component

changed the destination? Which relation witness justified the change? Which policy epoch and sink committed the effect? These objects can be retained off-chain with privacy controls, while only hashes or batch roots are externally anchored.

10.6 Relationship to information flow

CONTINUITY tracks selected provenance and taint but is not a full language-level information-flow control system. It focuses on structured effect fields and explicit transitions. CaMeL and Prompt Flow Integrity address broader control/data-flow separation [4, 5]; ARGUS tracks causal influence [7]. These mechanisms can serve as upstream producers of authenticated manifests, while CONTINUITY tests whether their output survives to finality. The composition contract does not substitute for their internal analyses.

11 Related Work

Prompt-injection evaluation. Early work demonstrated indirect prompt injection against application-integrated LLMs [1]. InjecAgent and AgentDojo provide systematic environments for tool-integrated and dynamic agent attacks [2, 3]. Those benchmarks primarily study whether an attacker can influence agent behavior and task outcomes. Our conformance suite begins from an adversarial proposal and instead targets discontinuities among downstream controls. The two evaluation styles are complementary: a natural-language benchmark estimates proposal pressure, while CONTINUITY tests whether a compromised proposal can cross a structural effect boundary.

Control/data isolation and privilege. CaMeL separates trusted control from untrusted data and constrains capabilities through program analysis [4]. Prompt Flow Integrity combines execution isolation, secure data handling, and privilege control [5]. IsolateGPT isolates executable agent components [11], and Progent provides programmable privilege control for tools [6]. These works motivate least authority and deterministic enforcement. Our focus is the interface theorem: what authenticated fields and guarantees must be preserved when such a control hands the action to a different policy, adapter, or sink.

Policy enforcement. Formal policy work translates policy into enforceable runtime constraints and evaluates practical agent systems [12]. The OWASP Agent Control Standard defines runtime interception and policy-decision hooks [13]. CONTINUITY does not compete with a policy language or hook API. It supplies a proof-carrying transition model in which a policy decision is one stage guarantee, and later components must preserve the facts on which that decision depended.

Provenance and execution integrity. ARGUS reconstructs causal runtime provenance and audits whether proposed actions are supported by trusted task information [7]. CXI binds field authority, typed releases, exact effects, and invocation authority at a protected sink [8]. Our design adopts the importance of field-level provenance, validated releases, and exact effect binding. The distinct contribution is composition across heterogeneous controls: trusted root enforcement, role-bound component contracts, signed before/after transition receipts, independently checked semantic relations, and a fault-injection methodology for context loss between provenance, policy, adapter, and finality layers.

Protocols and authorization. MCP specifies tool discovery and invocation and cautions that untrusted tool metadata is not itself authoritative [14, 15]. OAuth security guidance similarly emphasizes audience, least privilege, sender constraints, freshness, and replay resistance [21]. CONTINUITY binds these concepts to a concrete action and task witness but leaves transport and user authentication to existing protocols.

Compositional and supply-chain assurance. Assume-guarantee reasoning has a long history in compositional specifications [9]. Software supply-chain frameworks such as in-toto and TUF authenticate authorized transformations and compromise-resilient metadata rather than trusting only the final artifact [16, 17]. CONTINUITY applies a related insight to agent execution: each security-sensitive transformation must be attributable to an authorized role and satisfy an explicit relation before the final action is trusted.

Positioning. The paper does not claim novelty for digital signatures, provenance, capabilities, policy enforcement, typed releases, finality, JSON canonicalization, or assume-guarantee reasoning individually. It contributes a concrete end-to-end composition property, an implementation that enforces previously omitted root/release/-transform assumptions, and an executable taxonomy of cross-control discontinuities.

12 Limitations

Trusted roots are deployment inputs. The prototype now enforces trusted ingress, root-grant issuers, root bounds, and role-specific validators. It does not provision those roots or decide which organization should be trusted. Compromise or malicious configuration of a root authority, provenance issuer, release validator, transformation validator, verifier, or mandatory finality sink can invalidate the corresponding guarantee. Key rotation, quorum authorization, hardware roots, and certificate-path validation are outside the prototype.

Integrity is not semantic correctness. CONTINUITY guarantees continuity of authenticated security facts and authorization decisions, not the truth of those facts. A trusted provenance, release, or transformation validator may itself produce an incorrect structured conclusion. In that case, CONTINUITY can faithfully preserve and enforce an incorrect conclusion. Semantic correctness of trusted validators therefore remains part of the trusted computing base and is orthogonal to end-to-end consequence integrity.

Context-manifest completeness. The verifier recomputes and authenticates the context-manifest digest. The surrounding agent runtime must ensure that this manifest covers all bytes and tool outputs shown to the planner. The prototype does not instrument a production model runtime or prove completeness of its context capture. A runtime that silently exposes additional attacker-controlled content violates the assumption.

Restricted transformation language. The implementation evaluates one security-relevant relation, alias resolution, and a small deterministic predicate registry. Real deployments need richer, versioned semantics for currency conversion, schema mapping, identifier resolution, aggregation, and declassification. General-purpose relation languages create their own soundness, decidability, and policy-review challenges.

No verified implementation. The proofs apply to the abstract model and conditions. The Python artifact is extensively regression-tested but not mechanically verified, constant-time, hardened, or formally shown to refine the model. Its deterministic JSON encoder is intentionally restricted and should be replaced by a standards-conformant canonicalizer in interoperable deployments.

Synthetic conformance benchmark. The suite contains 32 designed fault templates, four domains, and deterministic parameter variations. It is useful for falsifying missing invariants and validating the artifact, but it is neither exhaustive nor sampled from a real attack population. The 0/2,560 result must not be interpreted as an estimated universal failure probability. It also does not compare model quality because no LLM is invoked.

Simplified provider semantics. The finality world, nonce ledger, and idempotency store are in-memory and atomic. Real providers exhibit concurrency, retries, eventual consistency, non-idempotent side effects, and partial failures. Provider-specific transaction protocols and durable evidence are required for production.

Incomplete information-flow coverage. The system checks declared protected fields and effects. It does not eliminate covert channels, infer every semantically equivalent effect path, or prevent leakage through allowed textual

output, timing, aggregate queries, resource names, or malicious downstream provider behavior. Complete mediation is an engineering and governance obligation.

Human and semantic error. A valid user grant may authorize an ill-advised action. A human can approve a misleading request, and a trusted tool can return false data. CONTINUITY preserves authorization context; it does not guarantee that the authorized goal is wise, legal, or semantically correct.

Artifact integration. The reference pipeline models memory, policy, adapter, and finality boundaries but is not a production MCP, A2A, OWASP ACS, cloud-IAM, or blockchain integration. Those bindings require separate protocol profiles and interoperability testing.

13 Conclusion

Agent security controls do not compose automatically. A provenance label, policy decision, capability, or action-bound permit can be locally correct yet fail when a neighboring component drops its assumptions, changes the action under a different schema, accepts an unauthorized signer, or commits an effect under stale state. We formalized this problem as security-context discontinuity and defined end-to-end consequence integrity.

CONTINUITY demonstrates a concrete response: authenticate the root grant; bind each stage to an authorized identity and contract; resolve provenance at leaf paths; make releases source-, value-, predicate-, task-, and expiry-bound; require independently checked relation witnesses for legitimate transformations; and revalidate subject, action, policy, revocation, and replay state at finality. In the deterministic artifact, these checks contain all 128 modeled fault-domain classes while preserving benign execution and explicit escalation.

The broader lesson is architectural: planning may remain probabilistic and adversarial, but external consequence should be mediated by a small, deterministic, proof-carrying control plane whose guarantees remain continuous from instruction context to realized effect.

A Proofs

We state the proof obligations at the abstraction level of [section 5](#). Cryptographic assumptions are signature unforgeability and digest collision resistance; implementation refinement is not claimed.

Lemma 1 (Root boundedness). *If $\text{RootOK}(E_0, \Gamma)$ holds, then the principal, actor, task, policy, provenance/context commitments, authority, delegation, tool/server/effect identities, data class, and constrained fields in E_0 are authorized by an accepted root issuer.*

Proof. The signatures bind E_0 and Γ to their producers. Deployment role checks restrict those producers to trusted ingress and grant issuers. Equality checks bind the identity, task, policy, commitments, and nonce. Subset and membership checks bound authority, delegation, tools, servers, effects, and classification. Every field constraint is evaluated on the resolved value in E_0 . Expiry and revocation checks establish current validity. Hence no accepted root fact exceeds Γ under the stated cryptographic assumptions. $\square$

Lemma 2 (Transition preservation). *If $\text{StepOK}(E_i, \rho_i, E_{i+1}, C_i, B_i)$ holds, then every security-critical difference between E_i and E_{i+1} is either an authorized monotone reduction or satisfies a declared transformation relation with a valid witness; all G_i guarantees are true of E_{i+1} .*

Proof. The receipt and envelope signatures plus input/output digests bind the exact endpoints, authorized signer, role, contract, stage, and sequence. Recomputed $\Delta(E_i, E_{i+1})$ prevents an omitted changed path. For each changed path, contract verification rejects a preserved path or a path lacking a declared relation. A witnessed path is accepted only if the trusted issuer, relation, path, before/after digests, component, contract, principal, task, and expiry agree and the deterministic relation predicate is true. Independent monotonicity checks reject increased authority or delegation, decreased taint, and policy downgrade. Required and ensured predicates are evaluated concretely; G_i is published only if no stage error occurs. $\square$

Lemma 3 (Inductive context continuity). *Suppose $\text{RootOK}(E_0, \Gamma)$ and StepOK holds for all $i \in [0, n - 1]$. Then every security field in E_n is connected to an authorized root value by a sequence of preserved equalities, monotone restrictions, or valid transformation relations, and every downstream assumption is discharged by an authenticated root guarantee or a valid predecessor guarantee.*

Proof. By induction on i . The base case follows from lemma 1. For the inductive step, lemma 2 preserves the property for unchanged fields, adds only justified relation edges for transformed fields, and forbids unauthorized amplification. Guarantee publication is conditional on successful verification, so the available guarantee set contains only root guarantees or guarantees from valid prior steps. Therefore the property holds at E_{i+1} . $\square$

Lemma 4 (Finality binding). *If a finality sink commits e under permit π , then the actual caller, action, audience, policy state, grant state, expiry, nonce, and idempotency state match π immediately before the effect, and $e = \text{Realize}(\text{Canon}(A_n))$.*

Proof. The sink accepts only the configured permit signer and verifies its signature. It compares the explicit caller subject, sink audience, and actual action digest to the permit. It compares policy identity, digest, and epoch to current runtime state and rejects revoked grants or permits.

It checks expiry and consumes one-time or idempotency state before applying the action. Collision resistance prevents a different canonical action from sharing the checked digest. The outcome receipt binds the committed effect and resulting state. $\square$

Proof of theorem 1. By C1 and lemma 1, E_0 has an authenticated, bounded origin and authenticated provenance/context commitments. The release clause of C4 validates every protected field whose bound source is untrusted. By C2–C4 and lemma 3, all fields in the final envelope are causally connected to that origin through valid contract steps without unauthorized authority or data amplification. By C5, the permit binds the same canonical action justified by the bundle. By C6 and lemma 4, the sink commits only under current, unconsumed state and the actual action, subject, and audience match. C7 excludes an alternate protected path without such a sink. Concatenating the grant, manifests, releases, envelopes, receipts, transformation witnesses, permit, and outcome yields W_e with $\text{Verify}(W_e) = 1$ and $\text{Realize}(\text{Canon}(W_e)) = e$. Therefore ECI holds. $\square$

Proof of theorem 2. Because D depends on f , there exist E, E' identical on all fields except f such that $D(E) \neq D(E')$. Since q omits f , $q(E) = q(E')$. Any deterministic downstream function of $q(E)$ alone returns the same decision for both and is therefore wrong for at least one. It must fail closed for both or obtain an authenticated recovery of f to distinguish them. $\square$

Proof of theorem 3. Let f influence the effect but be absent from all fields jointly committed by admission and the permit. Choose authorized action A and A' differing only at f such that their effects differ. Because the checked commitments omit f , the same permit validates both checked projections. An adversary controlling the intervening representation can present A' at the sink. If the sink does not rebind the complete action, it commits the unauthorized effect. $\square$

Proof of theorem 4. Induct on the number of ordinary derivation steps. The base authority is bounded by the authenticated root. By construction of StepOK , each accepted non-grant step enforces that output authority is a subset of input authority, so it cannot add an element. At a verified grant edge, only the authority explicitly contained in that valid grant may be added. Hence any authority present after finitely many steps belongs to the root or a verified grant, not to natural-language derivation alone. $\square$

Proof of proposition 1. Let p be a reachable path in a protected effect-equivalence class that bypasses a compatible finality sink. The adversary selects p and invokes the effect without producing a witness accepted by that sink. An effect is realized without the required W_e , contradicting ECI. Correctness of other paths is irrelevant to reachability of p . $\square$

B Representative Object Schemas

The listings omit signatures and some identifiers for space. The implementation signs the full canonical unsigned dataclass.

B.1 Root grant

```
{
  "grant_id": "grant:finance:17",
  "issuer": "key:authority",
  "principal": "did:example:user:17",
  "actor": "spiffe://example/agent/finance/4",
  "task_root": "task:finance:17",
  "authority": ["payment.transfer"],
  "delegation_scope": ["payment.transfer:account:17"],
  "allowed_tool_ids": ["urn:mcp:finance:payments"],
  "allowed_server_ids": ["spiffe://finance/payments"],
  "field_constraints": [{
    "path": "/action/parameters/amount_cents",
    "predicate": {"predicate_id": "int_range",
                  "parameters": {"min": 1, "max": 1000000}}
  ]},
  "policy_epoch": 43,
  "provenance_root": "sha256:...",
  "context_root": "sha256:...",
  "expires_at": 1800000600
}
```

B.2 Field provenance and typed release

```
{
  "claim": {
    "path": "/action/parameters/amount_cents",
    "source_id": "source:external:17:2",
    "value_digest": "sha256:..."
  },
  "release": {
    "issuer": "key:release",
    "principal": "did:example:user:17",
    "task_root": "task:finance:17",
    "source_id": "source:external:17:2",
    "source_digest": "sha256:...",
    "target_path": "/action/parameters/amount_cents",
    "value_digest": "sha256:...",
    "predicate": {"predicate_id": "int_range",
                  "parameters": {"min": 1, "max": 20000}},
    "action_operation": "payment.transfer",
    "tool_id": "urn:mcp:finance:payments",
    "expires_at": 1800000300
  }
}
```

B.3 Transformation witness and receipt

```
{
  "witness": {
    "issuer": "key:directory",
    "relation_id": "alias_resolution",
    "field_path": "/action/destination",
    "before_digest": "sha256:...",
    "after_digest": "sha256:...",
    "component_signer": "key:adapter",
    "contract_id": "contract:adapter:2",
    "parameters": {"alias": "alias:finance:17",
                  "resolved": "bankacct:merchant:17"}
  },
  "receipt": {
    "stage": "adapter",
    "role": "adapter",
    "input_digest": "sha256:...",
    "output_digest": "sha256:...",
    "changed_fields": ["/action/destination", "/representation"],
    "transformation_witness_ids": ["witness:..."]
  }
}
```

B.4 Execution permit

```
{
  "principal": "did:example:user:17",
  "subject": "spiffe://example/agent/finance/4",
  "task_root": "task:finance:17",
  "grant_id": "grant:finance:17",
  "audience": "spiffe://example/finality/finance",
  "action_digest": "sha256:...",
  "bundle_digest": "sha256:...",
  "policy_id": "policy://example/runtime",
  "policy_epoch": 43,
  "nonce": "nonce:...",
  "idempotency_key": "task:...:sha256:...",
  "one_time": true,
  "expires_at": 1800000060
}
```

C Reproducibility and Reason Codes

C.1 Commands

```
python -m venv .venv
. .venv/bin/activate
python -m pip install -r requirements.txt
```

```
python -m pytest -q
python scripts/run_experiments.py --output results
python scripts/make_figures.py \
  --results results --output figures
cp figures/*.pdf paper/figures/
cd paper
latexmk -pdf -interaction=nonstopmode \
  -halt-on-error main.tex
```

Quick smoke mode uses:

```
python scripts/run_experiments.py \
  --quick --output results_quick
```

The full main suite is deterministic. Latency depends on host, Python, and cryptographic-library versions. The manifest records scenario and run counts; raw CSV files expose terminal states, effect counts, first/second outcomes, reason strings, and proof sizes.

C.2 Expected checks

A clean artifact run should report 30 passing tests. For the full system, `summary.csv` should contain 2,560 attack instances, 128 contained fault-domain classes, effect ASR 0, benign auto-completion 1, ambiguous escalation 1, and lifecycle correctness 1. The exact timing columns are not invariant.

C.3 Representative reason-code families

Reason family	Meaning
E_UNTRUSTED_ROOT E_ROOT_*_EXCEEDED	Chain origin not in trusted ingress set Root authority, scope, class, or field bound exceeded
E_PROVENANCE_VALUE_MISMATCH	Resolved leaf differs from signed manifest claim
E_UNRELEASED_FIELD	Untrusted protected field needs validation/escalation
E_INVALID_RELEASE	Source, value, predicate, task, tool, or expiry check failed
E_UNAUTHORISED_STAGE_SIGNER	Key is not configured for stage/contract
E_RECEIPT_PRODUCER_MISMATCH	Receipt signer differs from output producer
E_MISSING_TRANSFORM_WITNESS	Changed path lacks required relation evidence
E_TRANSFORM_*	Witness role, field, digest, relation, or subject invalid
E_GUARANTEE_FALSE E_ACTION_SUBSTITUTION	Contract output postcondition failed Sink action differs from permit-bound action
E_SUBJECT_SUBSTITUTION	Actual caller differs from permit subject
E_REVOKED_AT_FINALITY	Grant or permit revoked after admission
E_UNMEDIATED_PATH	Protected effect attempted without permit

C.4 Claim-to-artifact map

Claim	Artifact evidence
Root and role enforcement	<code>core.py</code> ; root/role regression tests
Leaf provenance	JSON Pointer tests; provenance substitution tests
Typed release	predicate/value/expiry tests and 300 utility cases
Legitimate transformation	alias witness tests; every normal trace
Effect containment	<code>raw_results.csv</code> , <code>by_fault.csv</code>
Ablations	<code>ablation_raw.csv</code> , <code>ablation_summary.csv</code>
Lifecycle correctness	first/second outcomes and effect-count columns
Performance	<code>performance.csv</code> , <code>scaling.csv</code>

References

- [1] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. *More than You’ve Asked For: A Comprehensive Analysis of Novel Prompt Injection Threats to Application-Integrated Large Language Models*. 2023. arXiv: [2302.12173 \[cs.CR\]](#).
- [2] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. “InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents”. In: *Findings of the Association for Computational Linguistics: ACL 2024*. 2024, pp. 10471–10506. DOI: [10.18653/v1/2024.findings-acl.624](#).
- [3] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramer. *AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents*. 2024. arXiv: [2406.13352 \[cs.CR\]](#).
- [4] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramer. *Defeating Prompt Injections by Design*. 2025. arXiv: [2503.18813 \[cs.CR\]](#).
- [5] Juhee Kim, Woohyuk Choi, and Byoungyoung Lee. *Prompt Flow Integrity to Prevent Privilege Escalation in LLM Agents*. 2025. arXiv: [2503.15547 \[cs.CR\]](#).
- [6] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. *Progent: Securing AI Agents with Privilege Control*. 2025. arXiv: [2504.11703 \[cs.CR\]](#).
- [7] Shihao Weng, Yang Feng, Jinrui Zhang, Xiaofei Xie, Jiongchi Yu, and Jia Liu. *ARGUS: Defending LLM Agents Against Context-Aware Prompt Injection*. 2026. arXiv: [2605.03378 \[cs.CR\]](#).
- [8] Igor Santos-Grueiro. *Context-to-Execution Integrity for LLM Agents*. 2026. arXiv: [2607.06000 \[cs.CR\]](#).
- [9] Martin Abadi and Leslie Lamport. “Composing Specifications”. In: *ACM Transactions on Programming Languages and Systems* 15.1 (1993), pp. 73–132. DOI: [10.1145/151646.151649](#).
- [10] Paul C. Bryan and Kris Zyp. *JavaScript Object Notation (JSON) Pointer*. Tech. rep. RFC 6901. RFC Editor, 2013. DOI: [10.17487/RFC6901](#).
- [11] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. *IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems*. 2024. arXiv: [2403.04960 \[cs.CR\]](#).
- [12] Nils Palumbo, Sarthak Choudhary, Jihye Choi, Guy Amir, Prasad Chalasani, and Somesh Jha. *Formal Policy Enforcement for Real-World Agentic Systems*. 2026. arXiv: [2602.16708 \[cs.CR\]](#).
- [13] OWASP GenAI Security Project. *Agent Control Standard*. OWASP specification. Accessed 2026-09-04. 2026.
- [14] Model Context Protocol Contributors. *Model Context Protocol Specification: Tools*. Protocol specification, revision 2026-07-28. Accessed 2026-09-04. 2026.
- [15] Model Context Protocol Contributors. *Model Context Protocol Security Best Practices*. Protocol security guidance, revision 2026-07-28. Accessed 2026-09-04. 2026.
- [16] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. “in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes”. In: *28th USENIX Security Symposium*. 2019, pp. 1393–1410.
- [17] Justin Samuel, Nick Mathewson, Justin Cappos, and Roger Dingledine. “Surviving a Compromise of the Update Process”. In: *19th USENIX Security Symposium*. 2010, pp. 494–504.

- [18] Jerome H. Saltzer and Michael D. Schroeder. “The Protection of Information in Computer Systems”. In: *Proceedings of the IEEE* 63.9 (1975), pp. 1278–1308. DOI: [10.1109/PROC.1975.9939](https://doi.org/10.1109/PROC.1975.9939).
- [19] Scott Rose, Oliver Borchert, Stu Mitchell, and Sean Connelly. *Zero Trust Architecture*. Tech. rep. SP 800-207. National Institute of Standards and Technology, 2020. DOI: [10.6028/NIST.SP.800-207](https://doi.org/10.6028/NIST.SP.800-207).
- [20] Anders Rundgren, Bryce Jordan, and Samuel Erdtman. *JSON Canonicalization Scheme (JCS)*. Tech. rep. RFC 8785. RFC Editor, 2020. DOI: [10.17487/RFC8785](https://doi.org/10.17487/RFC8785).
- [21] Torsten Lodderstedt, John Bradley, Andrey Labunets, and Daniel Fett. *Best Current Practice for OAuth 2.0 Security*. Tech. rep. RFC 9700. RFC Editor, 2025. DOI: [10.17487/RFC9700](https://doi.org/10.17487/RFC9700).
- [22] Simon Josefsson and Ilari Liusvaara. *Edwards-Curve Digital Signature Algorithm (EdDSA)*. Tech. rep. RFC 8032. RFC Editor, 2017. DOI: [10.17487/RFC8032](https://doi.org/10.17487/RFC8032).